

Lennard–Jones CUDA benchmark report

Nejc Krajšek, Rok Mušič

May 2026

We benchmarked the final code on 5000 simulation steps for $N = 1000, 2000, 4000$ and 8000 particles. The block-size sweep used GPU block sizes 64, 128, 256, 512 and 1024. For the final CPU vs GPU comparison we used block size 128, because it was the fastest choice in every tested case. We implemented all the requirements for the basic grades, but none of the optional ones for a higher grade.

Block-size sweep. Table 1 shows the mean GPU time for each tested block size. We found out that block size 128 is the best choice for all four particle counts. Block size 64 is very close, which means that the kernel already has enough parallel work in that range. From block size 256 onward the runtime gets worse, and at 1024 it gets much worse. We interpret this as a simple balance issue: very large blocks give each launch less flexibility, so the GPU has a harder time keeping enough useful work active at the same time. For 8000 particles, we ran only one measurement for each block size, and 128 was still the fastest choice.

Table 1: Mean GPU runtime [s] for the block-size sweep at 5000 steps. Best time in each row is bold.

N	64	128	256	512	1024
1000	2.783	2.738	2.864	3.676	6.590
2000	4.707	4.688	4.813	5.840	10.551
4000	8.545	8.459	8.720	10.298	18.244
8000	16.026	15.972	16.614	19.335	33.439

Final timing results and speed-up. Table 2 gives the final CPU and GPU timings and the resulting speed-up. We found out that the GPU is already faster at $N = 1000$ by $11.49\times$, and that the advantage grows strongly with problem size, reaching $103.80\times$ at $N = 8000$. This trend is stable across the whole range: every time we doubled N , the CPU time increased by about $3.61\text{--}3.88\times$, while the GPU time increased by only about $1.71\text{--}1.89\times$. As a result, the speed-up itself increased by about $2.05\text{--}2.11\times$ for each doubling of N .

Table 2: Final benchmark results at 5000 steps using GPU block size 128.

N	CPU time [s]	GPU time [s]	Speed-up
1000	31.460 ± 0.025	2.739 ± 0.002	$11.49\times$
2000	113.443 ± 0.016	4.691 ± 0.002	$24.18\times$
4000	427.440 ± 0.047	8.457 ± 0.002	$50.54\times$
8000	1658.600 ± 0.000	15.979 ± 0.003	$103.80\times$

Figure 1 shows the same data on a log–log scale together with simple power-law fits. We found out that the CPU measurements follow an empirical trend close to $N^{1.91}$, which matches the expected strong growth of the all-pairs force computation ($O(n^2)$ in theory). The GPU fit is much flatter, about $N^{0.85}$ over the measured range. We do not claim that the true complexity changed; instead, this shows that the GPU overhead is better hidden as the problem gets larger (a similar trend to what we observed in the previous homework (a similar trend to what we observed in the previous homework)). The speed-up plot on the right confirms the same story in a direct way: larger systems use the GPU more effectively.

Correctness check. We also checked energy conservation. The mean relative total-energy drift stayed below 1.73×10^{-4} for every reported CPU and GPU case. The absolute drift grows with N , but the relative drift stays small and the CPU and GPU values stay close to each other. We therefore found no sign that the CUDA version changes the physical behaviour in a meaningful way over the measured 5000-step runs. As required, we also stored a separate final state visualization outside the report.

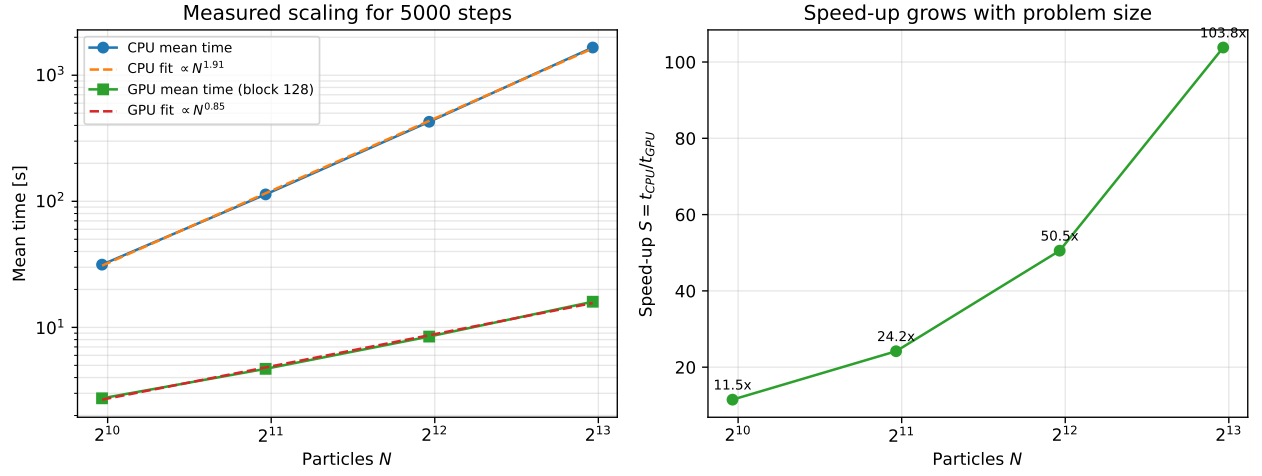


Figure 1: Left: measured CPU and GPU scaling with simple empirical power-law fits. Right: speed-up increases steadily with particle count.

Conclusion. We found out that the most important practical choice was the GPU block size, and that 128 threads per block was the best setting in all tested cases. With that choice, the GPU gave a clear speed-up already at 1000 particles and the benefit grew quickly with larger systems. The measured data therefore supports the main conclusion of our work: for this implementation, the GPU version is not only correct, but also much more effective than the CPU baseline once the particle count becomes moderately large.